“FARM ASSIST”

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

K. BHARGAVI	**: 2520030074**
M. SIRI CHANDANA	**: 2520030603**
G. NANDINI	**: 2520030300**

Team Number: 20
Section: 2

Under the Guidance of

Dr. S. Vinay Kumar
Associate Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **"FARM ASSIST"** is a record of Bonafide work of **K. BHARGAVI – 2520030074**, **M. SIRI CHANDANA – 2520030603**, **G. NANDINI – 2520030300** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

K. BHARGAVI – 2520030074

M. SIRI CHANDANA – 2520030603

G. NANDINI – 2520030300

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled **"FARM ASSIST"** is a bonafide work done and submitted by **K. BHARGAVI – 2520030074**, **M. SIRI CHANDANA – 2520030603**, **G. NANDINI – 2520030300** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27**.

Signature of the Guide
(Dr. S. Vinay Kumar)

Signature of the HOD
(Dr. P. Venkateshwara Rao)

Signature of the External Examiner

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to our project guide, Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering, for his valuable guidance, constant encouragement and patient support throughout the development of this project. His suggestions on choosing suitable algorithms and on presenting the work clearly helped us a great deal in shaping Farm Assist into a complete application.

We are thankful to Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the facilities and the academic environment needed to carry out this work. We also thank all the faculty members of the department for the knowledge they shared in the Data Structures and Algorithms courses, which formed the base of this project.

Finally, we thank our parents, friends and classmates for their support and for the useful discussions we had while testing the application. Their feedback helped us to improve the project.

K. BHARGAVI – 2520030074

M. SIRI CHANDANA – 2520030603

G. NANDINI – 2520030300

Abstract

Farmers and agriculture students often need quick answers to simple questions: which crop suits which season, what diseases and pests affect it, which fertilizer it needs, and how limited resources such as water, land and travel time can be used well. This information is usually scattered across books, notes and websites, and planning tasks are done by guess work. Our project, Farm Assist, is a console based application written in Java that brings crop knowledge and basic farm planning tools into a single program.

Farm Assist reads crop information from plain text files kept in a corpus folder and builds an in-memory knowledge base from it. A universal search accepts a crop name, a disease, a pest or any other keyword. It uses the Z-algorithm for exact crop names, the Knuth-Morris-Pratt algorithm to search inside every field, and Edit Distance to suggest the nearest word when the user makes a spelling mistake. A benchmark module compares Naive, KMP, Z-algorithm and Rabin-Karp searching on the same corpus. For planning, the application provides water distribution using the Edmonds-Karp maximum flow algorithm, crop-to-field assignment using bipartite matching, the shortest field-visit route using bitmask dynamic programming, a field inspection plan using a vertex cover 2-approximation, and a Crops of the Day feature that uses reservoir sampling.

The whole engine is built from basic arrays and our own helper classes, without using the java.util collections. The experiments show that all four string-search algorithms report identical matches, that KMP needs about 2n comparisons while the naive method grows as n×m in the worst case, and that the 2-approximation never exceeded twice the optimal number of fields to inspect. Farm Assist therefore shows how the algorithms studied in the course can be combined into one practical and easy to use tool for agriculture.

Table of Contents

			Page No.
	Acknowledgement		
	Abstract		
	Table of Contents		
	List of Figures		
1	Introduction	1.1 Overview	1-4
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives	
		1.5 Related Work and Research Gaps	
		1.6 Organisation of the Report	
2	System Requirements	2.1 Functional Requirements	5-7
		2.2 Non-Functional Requirements	
		2.3 Hardware and Software Specifications	
		2.4 Input Data (Corpus) Format	
3	System Design	3.1 High-level Architecture Diagram	8-11
		3.2 Description of Each Layer	
		3.3 Algorithm Selection	
		3.4 Search Workflow	
		3.5 Crop-Field Assignment as Max Flow	
		3.6 Data Structures Used	
4	Implementation	4.1 Module-wise Implementation	12-17
		4.2 Menu and Input Handling Logic	
		4.3 Complexity Summary	
5	Results	5.1 Experimental Setup	18-25
		5.2 Knowledge Search	
		5.3 String-Search Benchmark	
		5.4 Resource Planning Modules	
		5.5 Discussion	
6	Challenges & Limitations	6.1 Issues Faced during Development	27-28
		6.2 Solutions Applied	
		6.3 Limitations of the System	
7	Conclusion & Future Work	7.1 Conclusion	29-30
		7.2 Future Work	
		7.3 Skills Learned during Development	
	References		31
	Appendix A: Source Code		33

List of Figures

1.1	Module overview of Farm Assist	2
3.1	High-level architecture of Farm Assist	8
3.2	Flow chart of the universal search operation	10
3.3	Flow network used for crop-to-field assignment	11
5.1	Main menu and list of crops loaded from the corpus	18
5.2	Exact crop-name search (Z-algorithm)	19
5.3	Keyword search across all fields (KMP)	20
5.4	Fuzzy suggestion for a misspelt query (Edit Distance)	21
5.5	Output of the string-search benchmark	21
5.6	Naive vs. KMP vs. Z vs. Rabin-Karp in the worst case: (a) comparisons, (b) running time	22
5.7	Water distribution result (Max Flow)	23
5.8	Crop-to-field assignment result (Bipartite Matching)	24
5.9	Field visit route (Bitmask DP)	24
5.10	Field inspection planner (Vertex Cover 2-approximation)	25
5.11	Crops of the day (Reservoir Sampling)	25
A.1	Source code of Main.java (part 1 of 3)	33
A.2	Source code of Main.java (part 2 of 3)	34
A.3	Source code of Main.java (part 3 of 3)	34
A.4	Source code of CorpusReader.java	35
A.5	Source code of KnowledgeBase.java (part 1 of 2)	35
A.6	Source code of KnowledgeBase.java (part 2 of 2)	36
A.7	Source code of CropRecord.java	36
A.8	Source code of IntList.java	37
A.9	Source code of Util.java	37
A.10	Source code of FileSearcher.java (part 1 of 4)	38
A.11	Source code of FileSearcher.java (part 2 of 4)	39
A.12	Source code of FileSearcher.java (part 3 of 4)	40
A.13	Source code of FileSearcher.java (part 4 of 4)	41
A.14	Source code of NaiveSearch.java	41
A.15	Source code of KMP.java (part 1 of 2)	42
A.16	Source code of KMP.java (part 2 of 2)	42
A.17	Source code of ZAlgorithm.java	43
A.18	Source code of RabinKarp.java (part 1 of 2)	44
A.19	Source code of RabinKarp.java (part 2 of 2)	44
A.20	Source code of EditDistance.java	45
A.21	Source code of Benchmark.java (part 1 of 2)	45
A.22	Source code of Benchmark.java (part 2 of 2)	46
A.23	Source code of MaxFlow.java (part 1 of 3)	46
A.24	Source code of MaxFlow.java (part 2 of 3)	47
A.25	Source code of MaxFlow.java (part 3 of 3)	47

A.26 Source code of CropFieldMatcher.java (part 1 of 2) 48
A.27 Source code of CropFieldMatcher.java (part 2 of 2) 48
A.28 Source code of RoutePlanner.java (part 1 of 2) 49
A.29 Source code of RoutePlanner.java (part 2 of 2) 49
A.30 Source code of FieldInspector.java (part 1 of 2) 50
A.31 Source code of FieldInspector.java (part 2 of 2) 50
A.32 Source code of ReservoirSampler.java 51

1. INTRODUCTION

1.1 Overview

Agriculture is the main source of income for a large part of the population, and good decisions on a farm depend on having the right information at the right time. A farmer has to know the season and soil that suit a crop, the diseases and pests that may attack it, the fertilizer it needs and the amount of water it requires. Along with this knowledge, the farmer also has to plan things like how to share water between fields, which crop to plant in which field and in what order the fields should be visited. Most of these questions can be answered well using the standard tools of computer science. A data structure is a way to organize and store data that is usually chosen for efficient access to data, and an algorithm is the sequence of steps that works on it [1].

Farm Assist is a Java console application that brings these ideas together. It keeps crop information in plain text files, loads them into a knowledge base and lets the user search it in many ways. Searching uses classical string matching algorithms such as Knuth-Morris-Pratt [2], Rabin-Karp [3] and the Z-algorithm [4], and spelling mistakes are handled with the edit distance of Wagner and Fischer [5], which is based on the Levenshtein distance [6]. The planning part of the project uses the maximum flow method of Ford and Fulkerson [7] in its Edmonds-Karp form [8], dynamic programming over subsets for the shortest visiting route [9], a 2-approximation for the NP-hard vertex cover problem [10] and reservoir sampling [11] for choosing random crops. Fig. 1.1 shows the modules of the application.

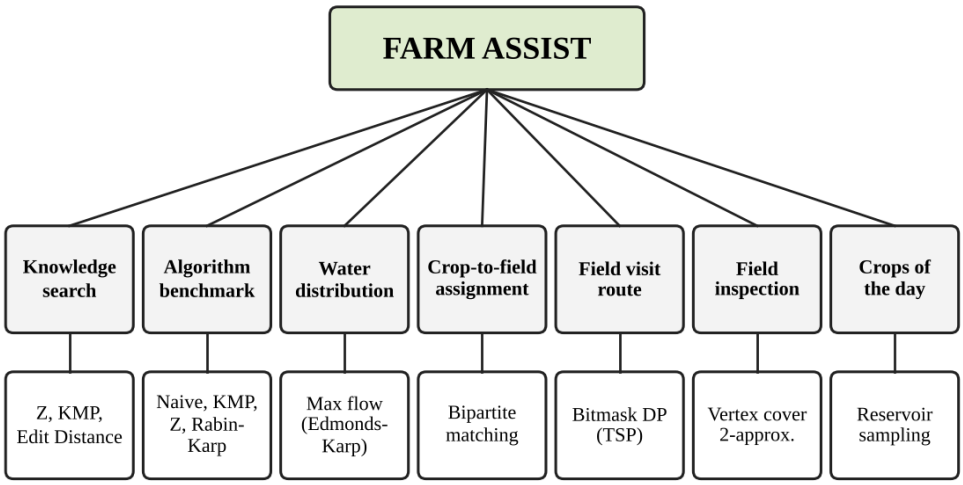


Fig. 1.1 Module overview of Farm Assist

Each module of Farm Assist is linked to one of the course outcomes of Data Structures and Algorithms-3: algorithm selection and analysis, pattern matching, dynamic programming, network flow, approximation algorithms and randomized algorithms. In this way the project is a working demonstration of the whole course and not just of one topic.

1.2 Problem Statement

Information about crops is spread over many books, leaflets and websites, and it is not easy for a farmer or student to find, for example, all the crops that are affected by a certain disease, or to correct a small spelling mistake in the search term. Planning problems such as sharing limited water, matching crops with suitable fields, finding a short route through several fields, and deciding which fields to inspect for disease are normally solved by guess work, which wastes time, water and fuel.

The problem addressed in this project is therefore to design and implement a single, easy to use application that (i) stores crop knowledge in a simple editable format and retrieves it quickly and tolerantly, and (ii) solves common farm planning problems with algorithms that are correct and efficient, while measuring and comparing the algorithms that are used.

1.3 Scope of the Project

The scope of Farm Assist covers the following:

- Loading any number of crop description files from a folder named corpus, where each crop is written in a simple Key: value format.
- Searching by crop name, disease, pest, soil, fertilizer or any other word that occurs in the corpus, with word boundary checking and spelling correction.
- Comparing four string search algorithms (Naive, KMP, Z-algorithm and Rabin-Karp) on the real corpus for correctness and cost.
- Solving five planning problems: water distribution, crop-to-field assignment, field visit route, field inspection planning and random crop selection.

The project does not include a graphical or web interface, live weather or market data, or machine learning based recommendation. These are possible future extensions and are listed in Section 7.2. The main challenge in the work was to build everything from basic arrays and our own helper classes, because the engine is not allowed to use the java.util collections or the java.util.Random class.

1.4 Objectives

- To build a knowledge base of crops from text files and to search it by any keyword.
- To select a suitable string matching algorithm by comparing Naive, KMP, Z-algorithm and Rabin-Karp experimentally.
- To handle spelling errors in user queries using Edit Distance.
- To model water distribution and crop-to-field assignment as maximum flow problems.
- To find the shortest round trip through the fields using bitmask dynamic programming.
- To plan field inspection with a vertex cover 2-approximation and to compare it with the exact optimum.
- To pick unbiased random crops from a list of any size using reservoir sampling.

1.5 Related Work and Research Gaps

Digital support for farmers is available mostly in the form of advisory websites, mobile applications and electronic books. These tools are useful, but in general they present the crop information as fixed pages or menus and rely on simple keyword matching, so a misspelt word returns nothing. The planning tools that exist, such as route planners and irrigation schedulers, are separate products and are not connected to the crop knowledge that the farmer is reading.

On the algorithm side, the individual techniques used in this project are well studied. Linear time string matching was introduced by Knuth, Morris and Pratt [2], randomized hashing based matching by Karp and Rabin [3], and the Z-function is described by Gusfield [4]. Network flow, bitmask dynamic programming for the travelling salesperson problem and approximation algorithms are standard topics in textbooks [1], [10]. What is missing in the literature and in the tools described above is a small, self-contained application that puts all of these techniques behind one menu, applies them to a real farm scenario and measures them against each other. Farm Assist is designed to fill this gap, mainly as a learning and demonstration platform.

1.6 Organisation of the Report

The rest of the report is organised as follows. Chapter 2 lists the functional and non-functional requirements and the hardware and software used. Chapter 3 presents the architecture and explains why each algorithm was chosen. Chapter 4 explains the implementation of every module using pseudocode. Chapter 5 presents the results of the experiments with figures and tables. Chapter 6 discusses the challenges and limitations, and Chapter 7 concludes the report and suggests future work.

2. SYSTEM REQUIREMENTS

This chapter describes what the system must do and the environment in which it runs.

2.1 Functional Requirements

The functional requirements describe the functions that Farm Assist performs. They are summarised in Table 2.1.

Table 2.1 Functional requirements

ID	Requirement	Algorithm used
FR1	Read all text files from the corpus folder and build the knowledge base of crop records.	Key: value parser
FR2	List the names of all crops that are loaded.	-
FR3	Search by crop name, disease, pest or any keyword and show the matching fields.	Z-algorithm, KMP
FR4	Suggest the closest word when the query has no match ("Did you mean ...?").	Edit Distance
FR5	Compare the four string-search algorithms on the corpus for a pattern given by the user.	Naive, KMP, Z, Rabin-Karp
FR6	Compute the maximum water that can be sent from a source to a field through a pipe network.	Edmonds-Karp
FR7	Assign crops to suitable fields so that the largest number of crops is planted together.	Bipartite matching (max flow)
FR8	Find the shortest round trip that visits every field or market starting from the farm.	Bitmask DP (TSP)
FR9	Choose the fields to inspect so that every pair of neighbouring fields is covered.	Vertex cover 2-approximation
FR10	Show k random crops of the day from the list.	Reservoir sampling
FR11	Validate every numeric input and repeat the question when the input is wrong.	-

2.2 Non-Functional Requirements

The non-functional requirements describe how well the system should work. They are given in Table 2.2.

Table 2.2 Non-functional requirements

Attribute	Requirement
Response time	A search over the corpus and every planning request on the allowed input sizes should finish in well under a second on a normal computer.

Attribute	Requirement
Usability	A numbered menu, clear prompts and repeated questions on wrong input, so that a user with no programming knowledge can use the program.
Scalability	The number of crops and fields is not fixed. Arrays grow automatically, so the corpus can contain any number of crops and files.
Reliability	All four string-search algorithms must return the same number of matches. Rabin-Karp verifies each hash hit to rule out collisions.
Portability	Runs on any operating system that has a Java Development Kit. No external library is needed.
Maintainability	Each algorithm is in its own class with comments that name the course outcome it covers. New crop data is added only by editing text files.
Constraint	The engine must not use java.util collections or java.util.Random. Own classes (IntList, CropRecord) and an own random number generator are used.

2.3 Hardware and Software Specifications

Farm Assist is a light program, so it does not need any special hardware. The specifications used for development and testing are given in Table 2.3 and Table 2.4.

Table 2.3 Hardware specifications

Component	Minimum	Used for testing
Processor	Any dual core processor	Multi-core x86-64 / ARM processor
Memory (RAM)	512 MB	4 GB or more
Disk space	50 MB free	Less than 5 MB used
Display / input	Console and keyboard	Console and keyboard

Table 2.4 Software specifications

Item	Specification
Language	Java (compiled and tested with OpenJDK 21) [12]
Operating system	Windows, Linux or macOS
Tools	javac compiler, any text editor or IDE, command line
Input data	Plain .txt files inside a folder named corpus
Libraries	Only java.io for input and output; no external libraries

2.4 Input Data (Corpus) Format

The knowledge base is created from text files. Every crop starts with the line CROP INFORMATION, followed by lines in the form Key: value. A value may continue on the next line. Any key can be used, so new fields such as Soil or Fertilizer can be added without changing the code. A sample record is shown below.

```
======================
CROP INFORMATION
======================
Crop Name: Rice
Season: Kharif
Soil: Clayey loam
Diseases: Blast, Bacterial leaf blight
Pests: Stem borer, Brown planthopper
Fertilizer: Urea, DAP
Water Requirement: 1200 mm
```

3. SYSTEM DESIGN

3.1 High-level Architecture Diagram

Farm Assist follows a simple layered design, shown in Fig. 3.1. The user talks only to the presentation layer. The algorithm layer contains the problem solving classes, and the data layer is responsible for reading the corpus and keeping it in memory. This separation means that an algorithm can be changed or tested without touching the menu or the file reading code.

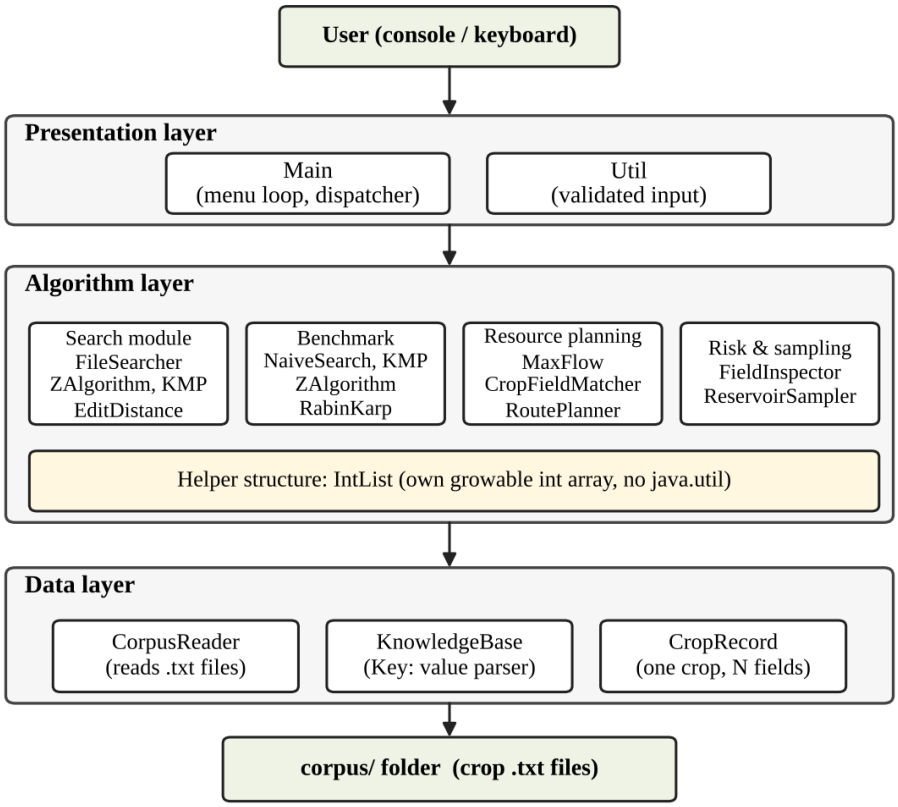


Fig. 3.1 High-level architecture of Farm Assist

3.2 Description of Each Layer

Presentation layer. The Main class shows the numbered menu in a loop and sends the choice to the right module. The Util class reads whole numbers and lists of numbers from the keyboard, and keeps asking when the input is not valid.

Algorithm layer. This layer has four groups of classes. The search group (FileSearcher, ZAlgorithm, KMP and EditDistance) answers queries on the knowledge base. The benchmark group (Benchmark, NaiveSearch, KMP, ZAlgorithm and RabinKarp) measures the string-search algorithms. The resource

planning group (MaxFlow, CropFieldMatcher and RoutePlanner) handles water, fields and routes. The risk and sampling group (FieldInspector and ReservoirSampler) handles disease risk and random selection. All of them use IntList, a growable array of integers written by us.

Data layer. CorpusReader reads each file line by line using a BufferedReader. KnowledgeBase cuts the whole text into crop sections at the marker CROP INFORMATION and turns each section into a CropRecord, which stores the field names (keys) and field contents (values) in two parallel arrays. The knowledge base also keeps the complete corpus in lower case, which is used by the benchmark.

3.3 Algorithm Selection

Each problem in the project was matched with the algorithm that suits it best. Table 3.1 gives the reasons.

Table 3.1 Algorithm selection

Problem	Algorithm	Why it is suitable	Time
Exact crop name	Z-algorithm [4]	Simple to code; one pass over pattern + text	O(n+m)
Keyword inside any field	KMP [2]	Never moves back in the text, so the worst case is still linear	O(n+m)
Spelling mistakes	Edit Distance [5]	Counts insertions, deletions and replacements between two words	O(m·n)
Algorithm comparison	Naive, KMP, Z, Rabin-Karp [3]	Shows the difference between O(n·m) and linear methods on real data	-
Water distribution	Edmonds-Karp [8]	BFS finds shortest augmenting paths, so it always terminates	$O(V \cdot E^2)$
Crop-to-field assignment	Matching as max flow	A unit capacity network gives the largest matching	$O(V \cdot E^2)$
Shortest field route	Bitmask DP [9]	Exact answer for up to 15 locations	$O(n^2 \cdot 2^n)$
Fields to inspect	Vertex cover 2-approx. [10]	Exact cover is NP-hard; maximal matching is within a factor of 2	O(E)
Crops of the day	Reservoir sampling [11]	Uniform choice of k items without needing to know the list size	O(N)

3.4 Search Workflow

The universal search is the main feature of the application. It works in three stages, as shown in Fig. 3.2. First the Z-algorithm checks whether the query is exactly a crop name, and if it is, the full details are shown. Otherwise KMP searches the query in every field of every crop, and a match is accepted only when it starts and ends at a word boundary, so that the query rice does not match the word price. If there is still no match, Edit Distance looks for the closest crop name or word in the corpus and offers it as a suggestion.

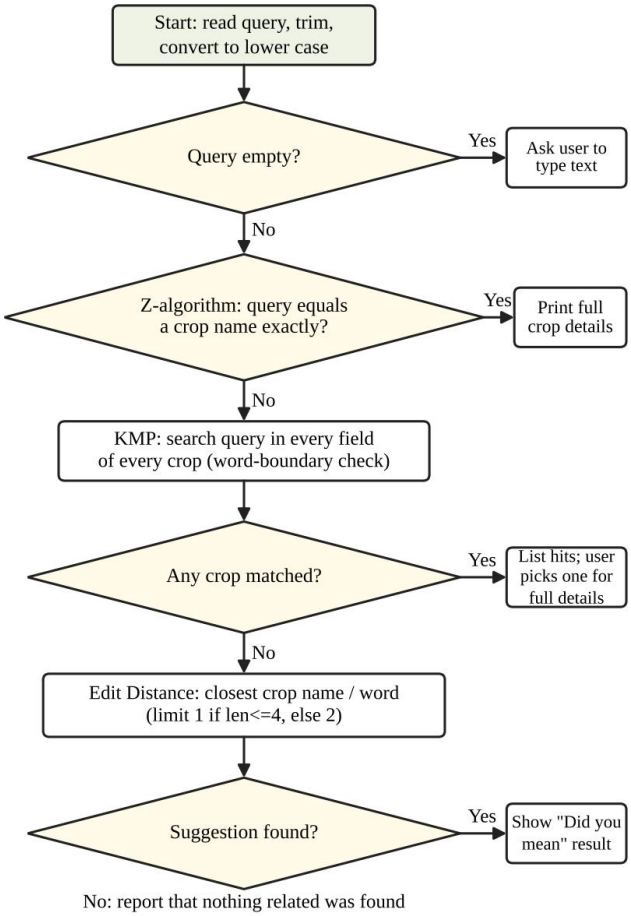


Fig. 3.2 Flow chart of the universal search operation

3.5 Crop-Field Assignment as Max Flow

Assigning crops to fields is a bipartite matching problem. It is solved by building a flow network, as shown in Fig. 3.3. A source S is joined to every crop with capacity 1, a crop is joined to every field that suits it with capacity 1, and every field is joined to a sink T with capacity 1. The value of the maximum flow from S to T is equal to the largest number of crops that can be planted at the same

time, and the edges that carry flow give the actual assignment. Because the same MaxFlow class is used for the water distribution and for this matching, the code is reused.

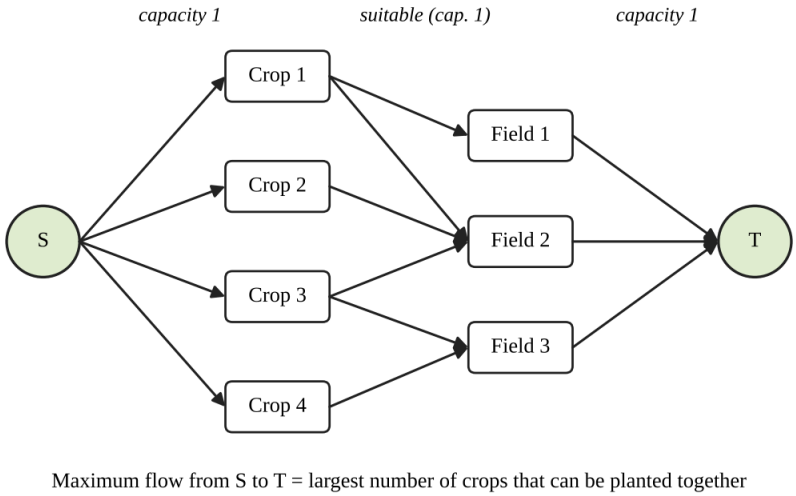


Fig. 3.3 Flow network used for crop-to-field assignment

3.6 Data Structures Used

- **IntList:** a growable integer array that doubles its size when it is full. It stores match positions without using java.util.ArrayList.
- **CropRecord:** two String arrays (keys and values) with a field counter. It is used instead of a map, so any field name is allowed.
- **Two dimensional int arrays:** adjacency matrices for flow networks and distance matrices for the route planner.
- **Bitmask tables:** dp[mask][i] and par[mask][i] arrays, where a bit of the mask shows whether a location is already visited.
- **Queue array:** a plain int array with front and rear indexes, used by the breadth first search in MaxFlow.

4. IMPLEMENTATION

This chapter explains the implementation of each module in pseudocode. The pseudocode follows the Java classes of the project, with n as the length of the text and m as the length of the pattern.

4.1 Module-wise Implementation

4.1.1 Corpus loading and knowledge base

Main reads every file of the corpus folder through CorpusReader and joins the text. KnowledgeBase then splits it at each marker and parses every section.

```
procedure BUILD-KNOWLEDGE-BASE(corpusText)
    corpusLower <- toLowerCase(corpusText)
    start <- indexOf("CROP INFORMATION")
    while start >= 0 do
        next <- indexOf("CROP INFORMATION", after start)
        section <- text from start up to next (or end)
        record <- PARSE(section)
        if record is not null then ADD(record)
        start <- next

procedure PARSE(section)
    for each line in section do
        skip blank lines and lines starting with === or ---
        colon <- indexOf(':', line)
        if 0 < colon <= 40 then
            record.addField(key before colon, value after)
        else
            record.appendToLastField(line)
    name <- value of the key "Crop Name" or "Name"
    return record
```

4.1.2 Naive string search

```
procedure NAIVE(text, pat)
    for i <- 0 to n - m do
        j <- 0
        while j < m and text[i + j] = pat[j] do
            comparisons++ ; j++
        if j = m then report i
```

4.1.3 Knuth-Morris-Pratt

```
procedure BUILD-LPS(pat)
    lps[0] <- 0 ; len <- 0 ; i <- 1
    while i < m do
        if pat[i] = pat[len] then len++ ; lps[i] <- len ; i++
        else if len > 0 then len <- lps[len - 1]
        else lps[i] <- 0 ; i++

procedure KMP(text, pat)
    lps <- BUILD-LPS(pat) ; i <- 0 ; j <- 0
    while i < n do
        if text[i] = pat[j] then
```

```
            i++ ; j++
            if j = m then report i - j ; j <- lps[j - 1]
        else if j > 0 then j <- lps[j - 1]
        else i++
```

4.1.4 Z-algorithm

The pattern, a separator character that is not present in the text, and the text are joined. Every position whose Z-value is at least m is a match.

```
procedure Z-FUNCTION(s)
    z[0..n-1] <- 0 ; left <- 0 ; right <- 0
    for i <- 1 to n - 1 do
        if i <= right then
            z[i] <- min(right - i + 1, z[i - left])
        while i + z[i] < n and s[z[i]] = s[i + z[i]] do z[i]++
        if i + z[i] - 1 > right then
            left <- i ; right <- i + z[i] - 1
    return z

procedure Z-SEARCH(text, pat)
    z <- Z-FUNCTION(pat + SEP + text)
    for i <- m + 1 to length(z) - 1 do
        if z[i] >= m then report i - m - 1
```

4.1.5 Rabin-Karp with double hashing

The rolling hash is computed with base 256 under two moduli, 1000000007 and 998244353. A window is checked character by character only when both hashes agree, which rules out false matches.

```
procedure RABIN-KARP(text, pat)
    p1 <- BASE^(m-1) mod M1 ; p2 <- BASE^(m-1) mod M2
    hp1,hp2 <- hashes of pat ; ht1,ht2 <- hashes of text[0..m-1]
    for i <- 0 to n - m do
        if hp1 = ht1 and hp2 = ht2 then
            if text[i..i+m-1] = pat then report i
        if i + m < n then
            ht1 <- ((ht1 - text[i]*p1) * BASE + text[i+m]) mod M1
            ht2 <- ((ht2 - text[i]*p2) * BASE + text[i+m]) mod M2
```

4.1.6 Edit Distance

```
procedure EDIT-DISTANCE(a, b)
    for i <- 0 to m do dp[i][0] <- i
    for j <- 0 to n do dp[0][j] <- j
    for i <- 1 to m do
        for j <- 1 to n do
            if a[i-1] = b[j-1] then dp[i][j] <- dp[i-1][j-1]
            else dp[i][j] <- 1 + min(dp[i-1][j],      // delete
                                     dp[i][j-1],      // insert
                                     dp[i-1][j-1])    // replace
    return dp[m][n]
```

4.1.7 Universal search

```
procedure SEARCH(kb, query)
    q <- trim(toLowerCase(query))
    if q is empty then ask for input ; return
    for each record r in kb do                      // stage 1
        if length(r.name) = length(q) and Z-SEARCH(r.name, q) found
            then PRINT-DETAILS(r) ; return
    hits <- PRINT-HITS(kb, q)                       // stage 2
    if hits is empty then                           // stage 3
        s <- SUGGEST(kb, q)
        if s is null then print "nothing found" ; return
        print "Did you mean s?"
        show record named s, or PRINT-HITS(kb, s)
    if hits not empty then ask which result to open in full

procedure CONTAINS-WORD(text, q)
    for each position s in KMP(text, q) do
        e <- s + length(q)
        if (s = 0 or text[s-1] is not letter/digit) and
           (e = length(text) or text[e] is not letter/digit)
            then return true
    return false

procedure SUGGEST(kb, q)
    if length(q) < 3 then return null
    limit <- 1 if length(q) <= 4 else 2
    for each crop name and each word of every field w do
        if |length(w) - length(q)| <= limit then
            d <- EDIT-DISTANCE(q, w)
            keep w with the smallest d <= limit
    return best
```

4.1.8 Water distribution (maximum flow)

Pipes are stored in a capacity matrix. The Edmonds-Karp method repeatedly finds the shortest path with spare capacity by breadth first search and pushes the largest possible flow along it.

```
procedure MAX-FLOW(graph, s, t)
    residual <- copy of graph ; flow <- 0
    while BFS(residual, s, t, parent) finds a path do
        pathFlow <- infinity
        for v <- t; v != s; v <- parent[v] do
            pathFlow <- min(pathFlow, residual[parent[v]][v])
        for v <- t; v != s; v <- parent[v] do
            residual[parent[v]][v] -= pathFlow
            residual[v][parent[v]] += pathFlow
        flow <- flow + pathFlow
    return flow
```

4.1.9 Crop-to-field assignment

```
procedure ASSIGN(crops, F fields)
    L <- min(number of crops, 8)
    S <- 0 ; T <- L + F + 1
    for each crop i do
        cap[S][crop i] <- 1
        read list of suitable fields
        for each suitable field f do cap[crop i][field f] <- 1
    for each field j do cap[field j][T] <- 1
    matched <- MAX-FLOW(cap, S, T)
```

```
    for each crop i do
        field <- the f with cap[crop i][f] = 1 and residual = 0
        print crop i -> field (or "not assigned")
```

4.1.10 Field visit route (bitmask DP)

Location 0 is the farm. The value dp[mask][i] is the shortest distance that starts at the farm, visits exactly the locations in mask and ends at location i. The array par stores the previous location so that the route can be printed.

```
procedure TSP(d, n)
    dp[all][all] <- INF ; dp[1][0] <- 0
    for mask <- 1 to 2^n - 1 do
        for i <- 0 to n - 1 do
            if i in mask and dp[mask][i] < INF then
                for j <- 0 to n - 1 do
                    if j not in mask then
                        nm <- mask OR (1 << j)
                        cost <- dp[mask][i] + d[i][j]
                        if cost < dp[nm][j] then
                            dp[nm][j] <- cost ; par[nm][j] <- i
    best <- min over i>0 of dp[2^n - 1][i] + d[i][0]
    rebuild the route from par starting at the best last location
```

4.1.11 Field inspection planner (vertex cover)

Fields are vertices and a pair of neighbouring fields that can pass a disease to each other is an edge. Choosing both ends of every edge that is still uncovered gives a cover of at most twice the optimal size. The exact optimum is also found by trying every subset (n is limited to 20), so that the ratio can be shown.

```
procedure APPROX-VERTEX-COVER(edges)
    chosen[0..n-1] <- false
    for each edge (a, b) do
        if not chosen[a] and not chosen[b] then
            chosen[a] <- true ; chosen[b] <- true
    return chosen

procedure EXACT-VERTEX-COVER(edges)
    opt <- n
    for mask <- 0 to 2^n - 1 do
        if bitCount(mask) < opt and mask covers every edge then
            opt <- bitCount(mask)
    return opt
```

4.1.12 Crops of the day (reservoir sampling)

Because java.util.Random is not allowed, a linear congruential generator is used with the multiplier 6364136223846793005 and the increment 1442695040888963407, seeded with the system time.

```
procedure RESERVOIR(stream, N, k)
    k <- min(k, N)
```

```
    for i <- 0 to N - 1 do
        if i < k then reservoir[i] <- stream[i]
        else
            j <- NEXT-INT(i + 1)           // random in 0..i
            if j < k then reservoir[j] <- stream[i]
    return reservoir

procedure NEXT-INT(bound)
    seed <- seed * 6364136223846793005 + 1442695040888963407
    return (seed >>> 33) mod bound
```

4.2 Menu and Input Handling Logic

The main method loads the corpus, builds the knowledge base and prints how many crops were loaded. If no crop is found, a warning asks the user to place the .txt files in the corpus folder. The program then repeats the menu until the user selects option 9.

```
load corpus ; kb <- BUILD-KNOWLEDGE-BASE(text)
loop
    print menu 1..9
    choice <- READ-INT("Enter your choice: ")
    case choice of
        1: SEARCH(kb, read line)       2: list all crop names
        3: BENCHMARK(kb, read line)    4: WATER-DISTRIBUTION
        5: ASSIGN(kb)                  6: TSP
        7: FIELD-INSPECTOR             8: RESERVOIR(names, count, k)
        9: exit
        otherwise: print "Invalid choice."

procedure READ-INT(prompt)
    repeat
        print prompt ; line <- read
        if end of input then exit program
        if line is a whole number then return it
        print "Please enter a whole number."
```

The Util class makes the application robust. A wrong number never stops the program, a blank line gives an empty list, and bad items in a list are skipped. Every module also checks its own limits, such as 2 to 15 locations for the route planner and 1 to 20 fields for the inspection planner.

4.3 Complexity Summary

Table 4.1 gives the time and space complexity of every module. Here V is the number of vertices and E the number of edges of a network, n the length of the text, and m the length of the pattern.

Table 4.1 Complexity of the modules

Module	Time	Extra space
Naive search	O(n·m) worst case	O(1)

Module	Time	Extra space
KMP	$O(n + m)$	$O(m)$
Z-algorithm	$O(n + m)$	$O(n + m)$
Rabin-Karp (double hash)	$O(n + m)$ average, $O(n \cdot m)$ worst	$O(1)$
Edit Distance	$O(m \cdot n)$	$O(m \cdot n)$
Max flow (Edmonds-Karp)	$O(V \cdot E^2)$ augmentation bound; each BFS $O(V^2)$ on a matrix	$O(V^2)$
Route planner (bitmask DP)	$O(n^2 \cdot 2^n)$	$O(n \cdot 2^n)$
Vertex cover approximation	$O(E)$	$O(V)$
Exact vertex cover (comparison)	$O(2^n \cdot E)$	$O(1)$
Reservoir sampling	$O(N)$	$O(k)$

5. RESULTS

5.1 Experimental Setup

All the modules were compiled with javac and run from the command line using OpenJDK 21. The knowledge base was built from a sample corpus of 10 crops (Rice, Wheat, Maize, Cotton, Groundnut, Sugarcane, Tomato, Chilli, Turmeric and Soybean) written in the format of Section 2.4. For the scaling experiment the same corpus was copied 100 times, which gave 1000 crop records and 224,100 characters. The worst-case experiment used a text of 100,000 characters. The figures in this chapter are the actual outputs of the program.

5.2 Knowledge Search

Fig. 5.1 shows the start of the application. The program reports that 10 crops were loaded and displays the menu. Option 2 lists all the crops.

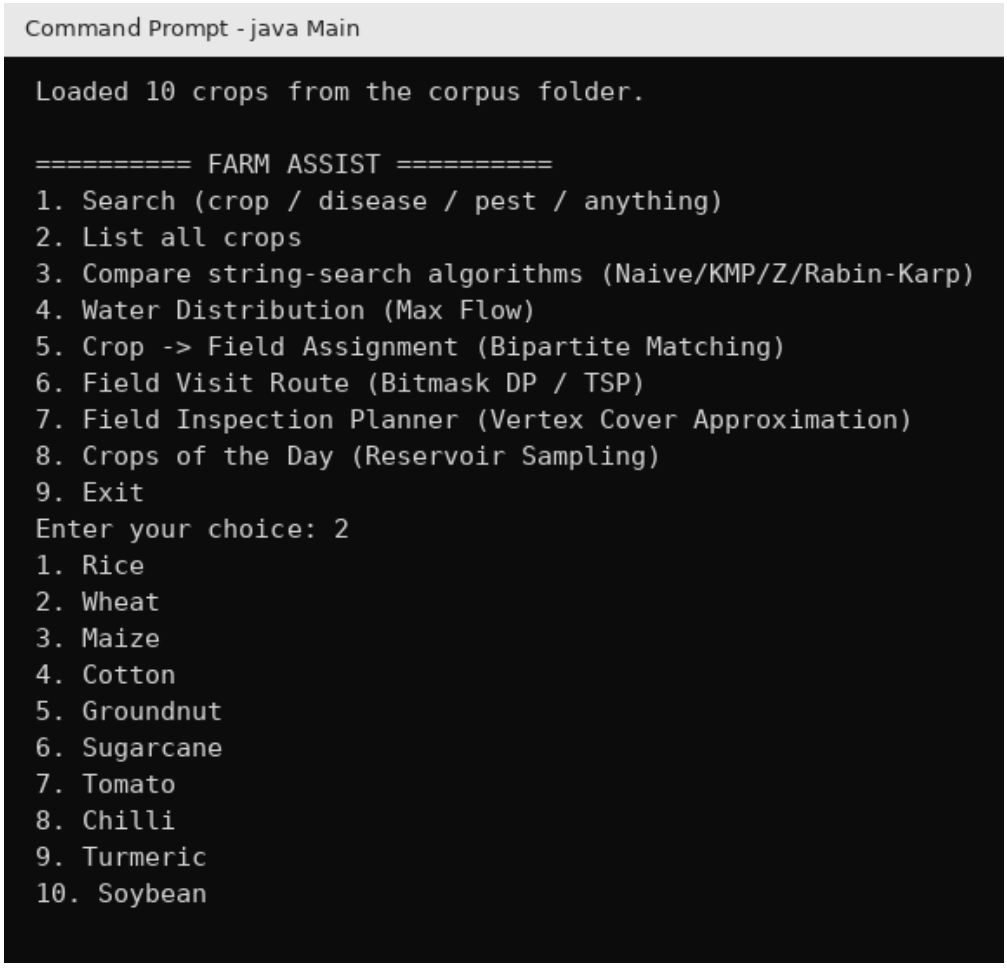


Fig. 5.1 Main menu and list of crops loaded from the corpus

When the query is exactly a crop name, for example rice, the Z-algorithm finds it and the complete record is shown (Fig. 5.2).

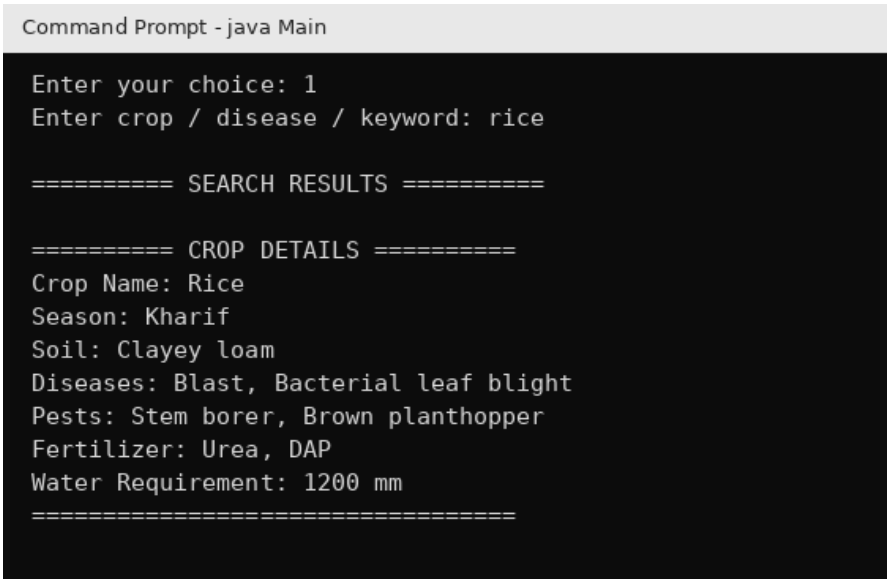


Fig. 5.2 Exact crop-name search (Z-algorithm)

When the query is a keyword, KMP searches every field of every crop. The query leaf curl was found in the Diseases field of three crops, and the user can open any of them in full (Fig. 5.3). Matches are accepted only at word boundaries.



Fig. 5.3 Keyword search across all fields (KMP)

When the query has a spelling mistake, Edit Distance proposes the closest word. The query whete is one edit away from Wheat, so the program suggests it and shows the details (Fig. 5.4).

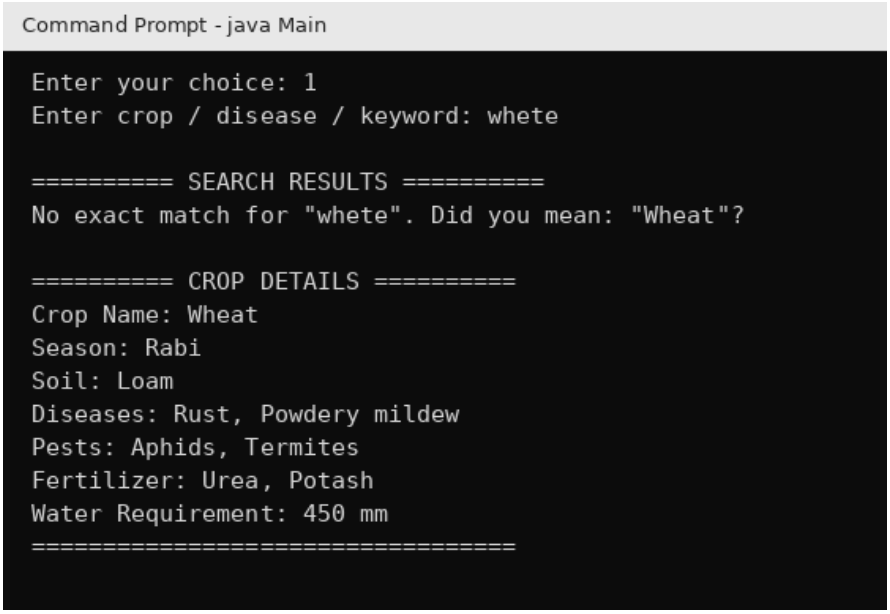


Fig. 5.4 Fuzzy suggestion for a misspelt query (Edit Distance)

5.3 String-Search Benchmark

The benchmark runs the four algorithms on the whole corpus and prints the number of matches, the time and the number of character comparisons (Fig. 5.5). For the pattern urea in the 2241 character corpus, all four algorithms found the same 6 matches, which confirms that they are correct.

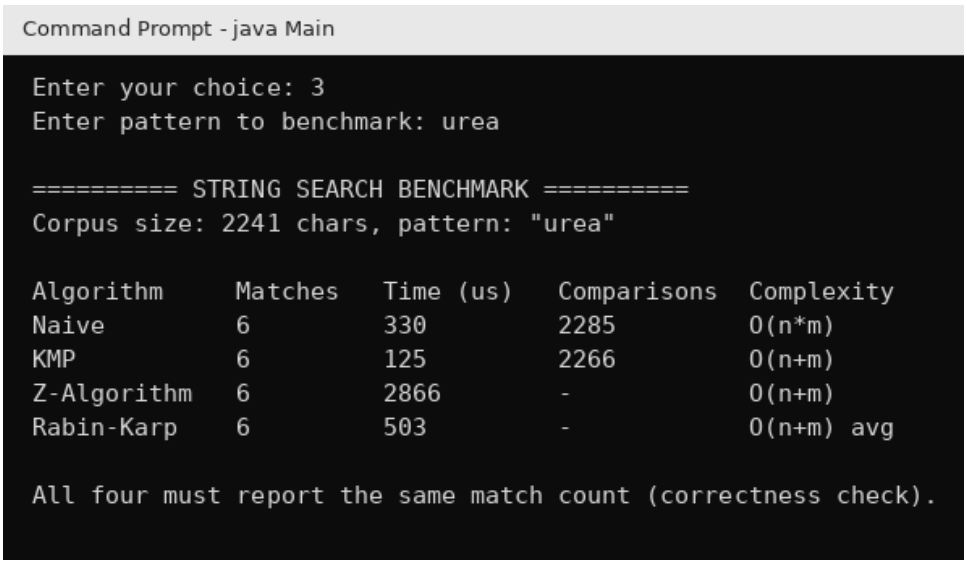


Fig. 5.5 Output of the string-search benchmark

Table 5.1 repeats the experiment on the larger corpus of 1000 crops. Again all four algorithms found the same number of matches. Because crop names and words in natural text differ in their first letter most of the time, the naive method also stops early and needs only slightly more comparisons than KMP. The running

times on such small inputs change from run to run because of the warm-up of the Java virtual machine, so the number of comparisons is the more reliable measure here.

Table 5.1 Benchmark on the 1000-crop corpus (224,100 characters)

Pattern	Matches (all four)	Naive comparisons	KMP comparisons
leaf curl	300	235,292	231,000
urea	600	228,797	226,600

To see the real difference between the algorithms, a worst-case text was used: 100,000 letters a and patterns made of m − 1 letters a followed by the letter b. Such a pattern matches almost everywhere for a long time before failing, so the naive method does the maximum amount of work. The results are in Table 5.2 and Fig. 5.6.

Table 5.2 Worst-case benchmark (n = 100,000)

m	Naive comp.	KMP comp.	Naive (µs)	KMP (µs)	Z (µs)	R-K (µs)
10	999,910	199,991	482	252	444	829
50	4,997,550	199,951	1,667	251	756	832
100	9,990,100	199,901	3,531	262	630	818
500	49,750,500	199,501	16,224	269	798	858
1000	99,001,000	199,001	29,401	260	480	881

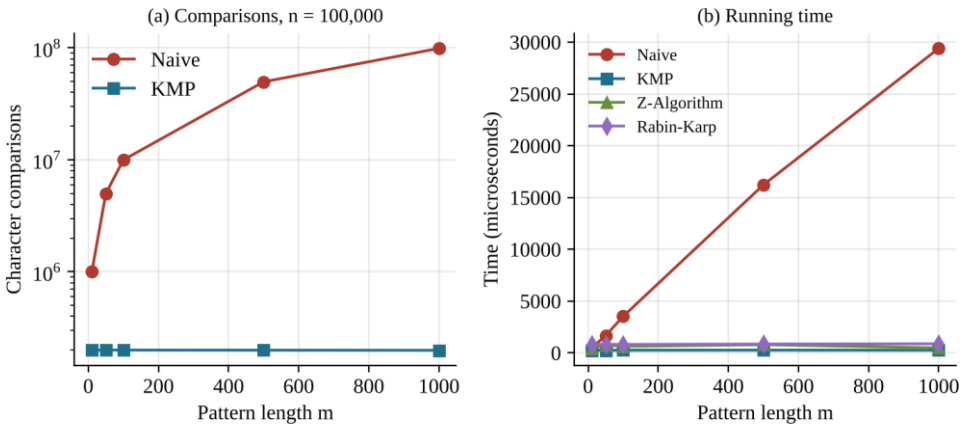


Fig. 5.6 Naive vs. KMP vs. Z vs. Rabin-Karp in the worst case: (a) comparisons, (b) running time

The naive method makes about n × m comparisons (99,001,000 for m = 1000) and its time grows in proportion to m, from 482 µs to 29,401 µs. KMP stays

at about 2n comparisons (below 200,000) for every pattern length, and its time is almost constant at about 260 µs. The Z-algorithm and Rabin-Karp are also independent of the pattern length. These results agree with the complexity analysis in Table 4.1 and justify the use of KMP for the keyword search of Farm Assist.

5.4 Resource Planning Modules

Water distribution. A network of 6 locations and 9 pipes was entered, with the reservoir at location 0 and the field at location 5. The program returned a maximum flow of 23 litres per minute (Fig. 5.7). This is the known answer for this standard textbook network [1], so the implementation is correct.

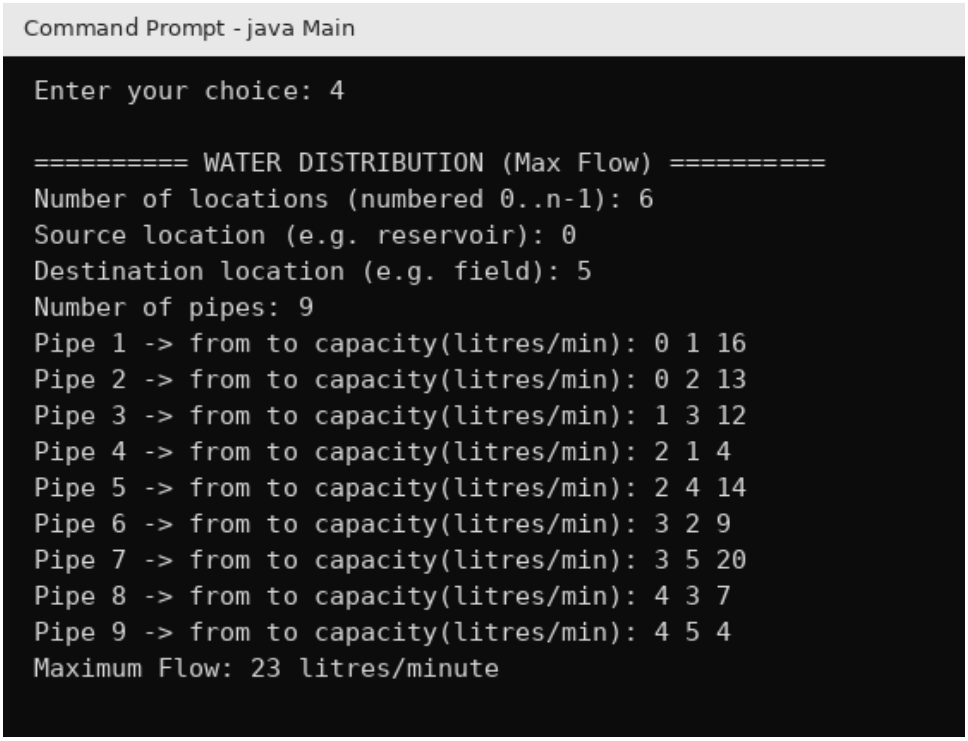


Fig. 5.7 Water distribution result (Max Flow)

Crop-to-field assignment. Eight crops were checked against four fields. The program found that 4 crops can be planted at the same time, which is the maximum possible because there are only four fields, and printed the assignment of each crop (Fig. 5.8).

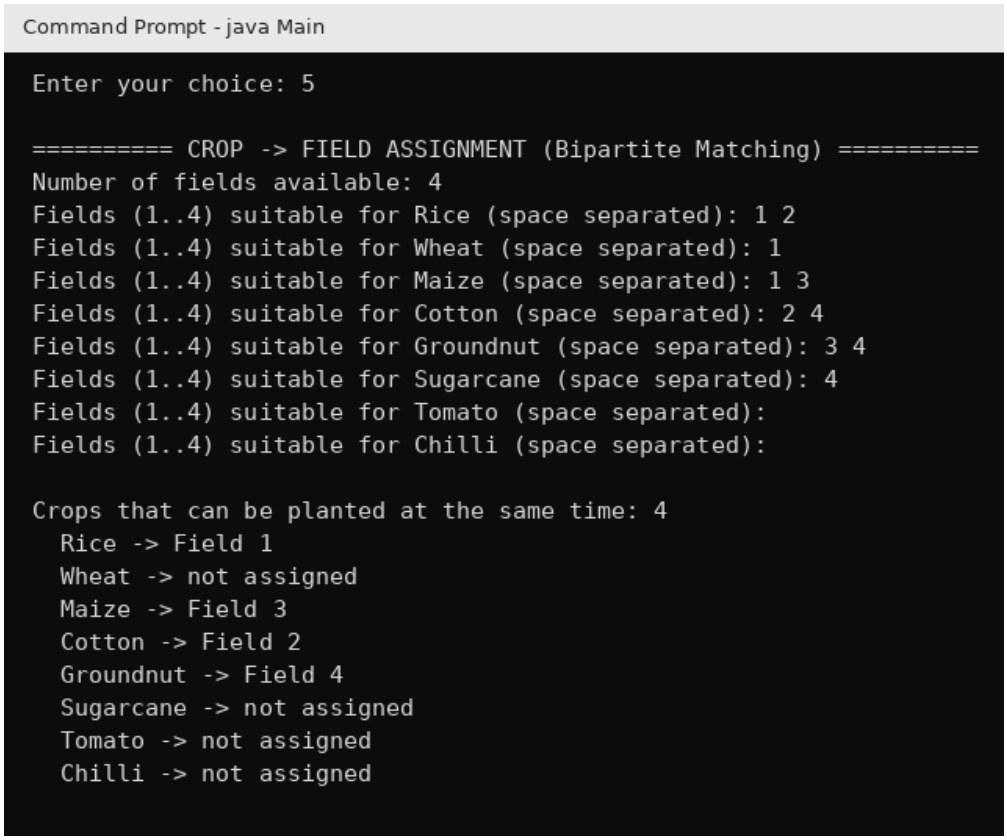


Fig. 5.8 Crop-to-field assignment result (Bipartite Matching)

Field visit route. For the farm and four fields with the distance matrix of Fig. 5.9, the bitmask DP found the route 0 → 2 → 4 → 3 → 1 → 0 with a total distance of 85. Adding the legs gives 15 + 20 + 15 + 25 + 10 = 85, and no other order of the four fields is shorter.

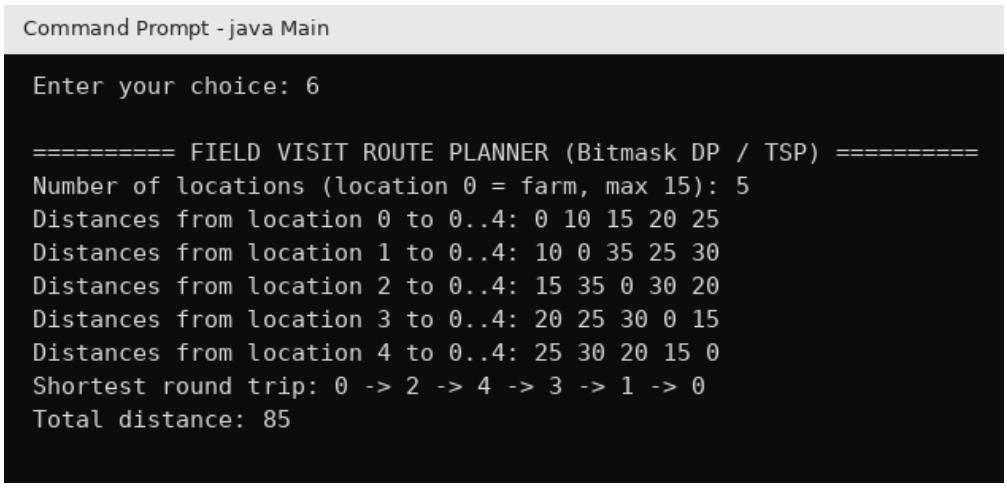


Fig. 5.9 Field visit route (Bitmask DP)

Field inspection. For 7 fields and 7 neighbouring pairs, the approximation selected 4 fields (0, 1, 4 and 5) while the exact optimum is 3, so the ratio is 1.33

(Fig. 5.10). In a second test with 6 fields joined in a ring, the approximation selected all 6 fields and the optimum was 3, giving the ratio 2.0. This is the worst case allowed by the theory, and it shows that the guarantee of at most twice the optimum holds and that it is tight. Table 5.3 summarises the two tests.

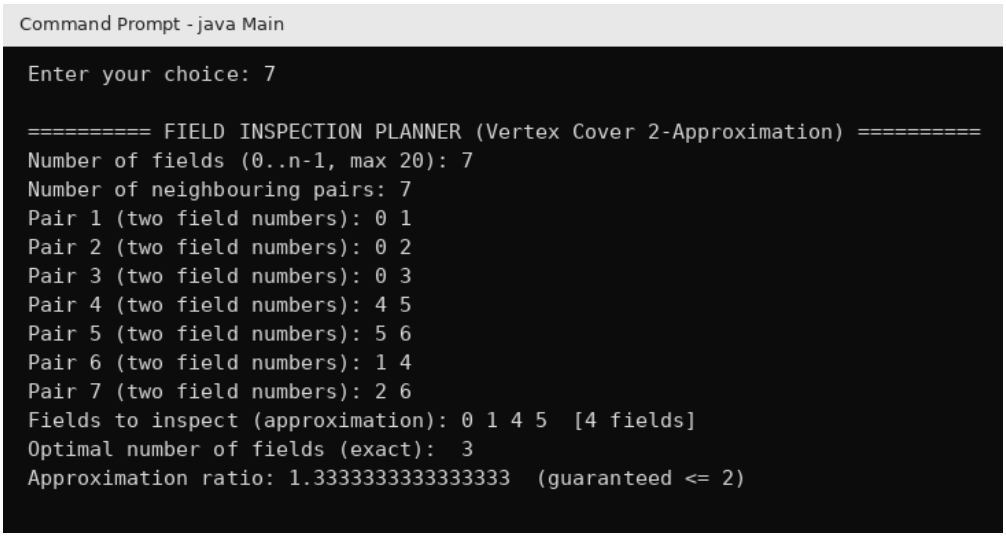


Fig. 5.10 Field inspection planner (Vertex Cover 2-approximation)

Table 5.3 Approximation versus exact vertex cover

Test	Fields	Pairs	Approximation	Optimal	Ratio
Test 1	7	7	4	3	1.33
Test 2 (ring)	6	6	6	3	2.00

Crops of the day. Asking for 3 crops returned Tomato, Turmeric and Chilli (Fig. 5.11). Since the generator is seeded with the system time, a different set is shown in each run, and every crop has the same chance of being chosen.



Fig. 5.11 Crops of the day (Reservoir Sampling)

5.5 Discussion

The experiments show that every module gives correct results. The four string-search algorithms always agree on the number of matches. The flow, route and inspection results agree with values that can be checked by hand. The benchmark shows clearly why linear time algorithms are preferred as the corpus

and the patterns grow. At the same time, the results show that on ordinary text with a small corpus the choice of algorithm makes little difference, which is why Farm Assist uses the simple Z-algorithm for whole names and KMP for the keyword search. The results given here are for the sample corpus. A bigger corpus with more crops, pests and diseases will give larger absolute numbers, but the same behaviour.

6. CHALLENGES & LIMITATIONS

6.1 Issues Faced during Development

- **No standard collections.** Because java.util is not allowed inside the engine, we had to build our own growable array (IntList) and use plain arrays that are enlarged by hand in KnowledgeBase and CropRecord.
- **No standard random generator.** java.util.Random could not be used for the Crops of the Day feature.
- **Free format of the corpus.** Crop files may have different field names and values that continue on several lines.
- **Wrong matches inside words.** A plain substring search for rice also matches words such as price.
- **Hash collisions in Rabin-Karp.** Two different windows can have the same hash value.
- **Memory growth of the dynamic programming.** The route planner stores $2^n \times n$ values, which grows very fast.
- **Invalid input.** Typing a letter where a number is expected, or leaving a line blank, could crash a console program.

6.2 Solutions Applied

- IntList doubles its array when full, and the same idea is used for the record arrays.
- A small linear congruential generator seeded with System.nanoTime() was written inside ReservoirSampler.
- The parser treats any line of the form Key: value (with the colon in the first 40 characters) as a field and any other line as a continuation of the previous value.
- KMP results are accepted only if the characters before and after the match are not letters or digits (word boundary check).
- Double hashing with two different moduli is used, and every hash match is verified character by character.
- The number of locations is limited to 15 for the route planner and 20 for the inspection planner, and the input is checked.

- Util.readInt repeats the question until a valid number is given, and readInts skips bad items.

6.3 Limitations of the System

- The application has a console interface only, and the user must type the numbers for networks, distances and field pairs by hand.
- The crop-to-field matching considers only the first 8 crops of the knowledge base.
- The route planner is limited to 15 locations and the exact vertex cover check to 20 fields, because both are exponential.
- The flow module uses an adjacency matrix, which needs memory proportional to V^2.
- Spelling correction allows only one or two edits and works on English spellings of words that are present in the corpus.
- The knowledge is only as good as the text files provided, and the data is not saved or updated by the program itself.
- The random generator is meant for sampling and is not suitable for security purposes.

7. CONCLUSION & FUTURE WORK

7.1 Conclusion

Farm Assist combines a searchable crop knowledge base with five planning tools in a single console application. The universal search finds crops by name or by any keyword and corrects spelling errors. The benchmark proves that Naive, KMP, Z-algorithm and Rabin-Karp give the same answers and that KMP keeps its cost at about 2n comparisons while the naive method reaches about n × m. The maximum flow module distributes water and assigns crops to fields, the bitmask DP gives the shortest visiting route, the vertex cover 2-approximation guides field inspection within twice the optimal effort, and reservoir sampling chooses crops at random fairly. All of this was achieved with our own array classes and without the java.util collections.

The project showed that the algorithms learned in the course are directly useful for a real problem. Its main limitation is that it works only on a console and on small inputs for the exponential modules, but the design is modular, and each part can be improved separately.

7.2 Future Work

- A graphical or web based interface, including a mobile application, with support for local languages.
- Reading the pipe network, distances and field neighbours from files or from a map instead of typing them.
- Using a heuristic or branch and bound method for the route planner so that more than 15 locations can be handled.
- Adding weather, soil test and market price data to give crop recommendations.
- Using a graph with adjacency lists for the flow module to save memory on large networks.
- Saving user farms and plans in a database, and adding image based disease identification.

7.3 Skills Learned during Development

- Designing and analysing algorithms: pattern matching, dynamic programming, network flow, approximation and randomization.
- Modelling a real problem (assigning crops to fields) as a standard graph problem.
- Writing programs in Java with our own data structures, and testing them with correct and worst-case inputs.
- Measuring and comparing the performance of algorithms, and understanding why timings of small inputs can mislead.
- Writing documentation, pseudocode and reports, and working together as a team.

REFERENCES

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.
2. D. E. Knuth, J. H. Morris and V. R. Pratt, “Fast pattern matching in strings,” SIAM Journal on Computing, vol. 6, no. 2, pp. 323–350, 1977.
3. R. M. Karp and M. O. Rabin, “Efficient randomized pattern-matching algorithms,” IBM Journal of Research and Development, vol. 31, no. 2, pp. 249–260, 1987.
4. D. Gusfield, Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology. Cambridge, U.K.: Cambridge University Press, 1997.
5. R. A. Wagner and M. J. Fischer, “The string-to-string correction problem,” Journal of the ACM, vol. 21, no. 1, pp. 168–173, 1974.
6. V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions and reversals,” Soviet Physics Doklady, vol. 10, no. 8, pp. 707–710, 1966.
7. L. R. Ford and D. R. Fulkerson, “Maximal flow through a network,” Canadian Journal of Mathematics, vol. 8, pp. 399–404, 1956.
8. J. Edmonds and R. M. Karp, “Theoretical improvements in algorithmic efficiency for network flow problems,” Journal of the ACM, vol. 19, no. 2, pp. 248–264, 1972.
9. M. Held and R. M. Karp, “A dynamic programming approach to sequencing problems,” Journal of the Society for Industrial and Applied Mathematics, vol. 10, no. 1, pp. 196–210, 1962.
10. M. R. Garey and D. S. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness. San Francisco, CA, USA: W. H. Freeman, 1979.
11. J. S. Vitter, “Random sampling with a reservoir,” ACM Transactions on Mathematical Software, vol. 11, no. 1, pp. 37–57, 1985.

12. Oracle, “Java Platform, Standard Edition documentation.” [Online]. Available: https://docs.oracle.com/en/java/

APPENDIX A: SOURCE CODE

This appendix gives the complete Java source code of Farm Assist, class by class, as it was compiled and run for the experiments in Chapter 5.

Main.java (part 1 of 3)

```java
import java.io.BufferedReader;
import java.io.File;
import java.io.InputStreamReader;

public class Main {
    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new
            InputStreamReader(System.in));

        StringBuilder corpus = new StringBuilder();
        File[] files = new File("corpus").listFiles();
        if (files != null) {
            for (File f : files) if (f.isFile())
                corpus.append(CorpusReader.readFile(f));
        }
        KnowledgeBase kb = new KnowledgeBase(corpus.toString());
        System.out.println("Loaded " + kb.count + " crops from the corpus
            folder.");
        if (kb.count == 0) System.out.println("WARNING: no crops found. Put
            your .txt files in a folder named 'corpus'.");

        while (true) {
            System.out.println("\n========== FARM ASSIST ==========");
            System.out.println("1. Search (crop / disease / pest /
                anything)");
            System.out.println("2. List all crops");
            System.out.println("3. Compare string-search algorithms
                (Naive/KMP/Z/Rabin-Karp)");
            System.out.println("4. Water Distribution (Max Flow)");
            System.out.println("5. Crop -> Field Assignment (Bipartite
                Matching)");
            System.out.println("6. Field Visit Route (Bitmask DP / TSP)");
            System.out.println("7. Field Inspection Planner (Vertex Cover
                Approximation)");
            System.out.println("8. Crops of the Day (Reservoir Sampling)");
            System.out.println("9. Exit");
```

Fig. A.1 Source code of Main.java (part 1 of 3)

Main.java (part 2 of 3)

```
29
30              int choice = Util.readInt(br, "Enter your choice: ");
31              switch (choice) {
32                  case 1:
33                      System.out.print("Enter crop / disease / keyword: ");
34                      FileSearcher.search(kb, br.readLine(), br);
35                      break;
36                  case 2:
37                      for (int i = 0; i < kb.count; i++) System.out.println((i +
                            1) + ". " + kb.records[i].name);
38                      break;
39                  case 3:
40                      System.out.print("Enter pattern to benchmark: ");
41                      Benchmark.run(kb, br.readLine().trim());
42                      break;
43                  case 4:
44                      MaxFlow.runWaterDistribution(br);
45                      break;
46                  case 5:
47                      CropFieldMatcher.run(br, kb);
48                      break;
49                  case 6:
50                      RoutePlanner.run(br);
51                      break;
52                  case 7:
53                      FieldInspector.run(br);
54                      break;
55                  case 8:
56                      int k = Util.readInt(br, "How many crops? ");
57                      String[] names = new String[kb.count];
58                      for (int i = 0; i < kb.count; i++) names[i] =
                            kb.records[i].name;
59                      String[] pick = ReservoirSampler.sample(names, kb.count,
                            k);
60                      for (String s : pick) System.out.println(" * " + s);
61                      break;
```

Fig. A.2 Source code of Main.java (part 2 of 3)

Main.java (part 3 of 3)

```
62                  case 9:
63                      System.out.println("Exiting Farm Assist...");
64                      return;
65                  default:
66                      System.out.println("Invalid choice.");
67              }
68          }
69      }
70  }
```

Fig. A.3 Source code of Main.java (part 3 of 3)

CorpusReader.java

```java
import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.IOException;

public class CorpusReader {
    // Read one complete corpus file (no java.util.Scanner)
    public static String readFile(File file) {
        StringBuilder content = new StringBuilder();
        try (BufferedReader reader = new BufferedReader(new FileReader(file)))
             {
            String line;
            while ((line = reader.readLine()) != null) {
                content.append(line).append("\n");
            }
        } catch (IOException e) {
            System.out.println("Unable to read file: " + file.getName());
            return "";
        }
        return content.toString();
    }
}
```

Fig. A.4 Source code of CorpusReader.java

KnowledgeBase.java (part 1 of 2)

```java
// Parses the whole corpus into CropRecord objects (generic Key: value parser,
// so it works for any field name: Diseases, Pests, Soil, Fertilizer, ...)
public class KnowledgeBase {
    public CropRecord[] records = new CropRecord[16];
    public int count = 0;
    public String corpusLower = "";   // whole corpus, lowercase (used by
        benchmark)

    private static final String MARKER = "CROP INFORMATION";

    public KnowledgeBase(String corpusText) {
        corpusLower = corpusText.toLowerCase();
        int start = corpusText.indexOf(MARKER);
        while (start >= 0) {
            int next = corpusText.indexOf(MARKER, start + MARKER.length());
            String section = (next < 0) ? corpusText.substring(start) :
                corpusText.substring(start, next);
            CropRecord rec = parse(section);
            if (rec != null) add(rec);
            start = next;
        }
    }

    private void add(CropRecord r) {
        if (count == records.length) {
            CropRecord[] bigger = new CropRecord[count * 2];
            System.arraycopy(records, 0, bigger, 0, count);
            records = bigger;
        }
        records[count++] = r;
    }

    private static CropRecord parse(String section) {
        CropRecord rec = new CropRecord();
        String[] lines = section.split("\\r?\\n");
        for (String raw : lines) {
```

Fig. A.5 Source code of KnowledgeBase.java (part 1 of 2)

KnowledgeBase.java (part 2 of 2)

```
            String line = raw.trim();
            if (line.isEmpty() || line.startsWith("===") ||
                line.startsWith("---")) continue;
            if (line.equalsIgnoreCase("CROP INFORMATION")) continue;
            int colon = line.indexOf(':');
            if (colon > 0 && colon <= 40) {
                rec.addField(line.substring(0, colon).trim(),
                    line.substring(colon + 1).trim());
            } else {
                rec.appendToLastField(line);   // continuation of previous
                    value
            }
        }
        if (rec.fieldCount == 0) return null;
        for (int i = 0; i < rec.fieldCount; i++) {
            String k = rec.keys[i].toLowerCase().replace("_", "").replace(" ",
                 "");
            if (k.equals("cropname") || k.equals("name")) { rec.name =
                rec.values[i]; break; }
        }
        if (rec.name.isEmpty()) rec.name = rec.values[0];
        return rec;
    }

    public static boolean isNameKey(String key) {
        String k = key.toLowerCase().replace("_", "").replace(" ", "");
        return k.equals("cropname") || k.equals("name");
    }
}
```

Fig. A.6 Source code of KnowledgeBase.java (part 2 of 2)

CropRecord.java

```
// One "CROP INFORMATION" section: a list of Key: value fields
public class CropRecord {
    public String name = "";
    public String[] keys = new String[8];
    public String[] values = new String[8];
    public int fieldCount = 0;

    public void addField(String key, String value) {
        if (fieldCount == keys.length) {
            String[] k = new String[fieldCount * 2];
            String[] v = new String[fieldCount * 2];
            System.arraycopy(keys, 0, k, 0, fieldCount);
            System.arraycopy(values, 0, v, 0, fieldCount);
            keys = k;
            values = v;
        }
        keys[fieldCount] = key;
        values[fieldCount] = value;
        fieldCount++;
    }

    public void appendToLastField(String more) {
        if (fieldCount > 0) values[fieldCount - 1] = values[fieldCount - 1] +
            " " + more;
    }
}
```

Fig. A.7 Source code of CropRecord.java

IntList.java

```java
// Hand-built growable int array (java.util is not allowed inside the engine)
public class IntList {
    private int[] data = new int[16];
    private int size = 0;
    public void add(int v) {
        if (size == data.length) {
            int[] bigger = new int[size * 2];
            System.arraycopy(data, 0, bigger, 0, size);
            data = bigger;
        }
        data[size++] = v;
    }
    public int get(int i) { return data[i]; }
    public int size() { return size; }
    public int[] toArray() {
        int[] r = new int[size];
        System.arraycopy(data, 0, r, 0, size);
        return r;
    }
}
```

Fig. A.8 Source code of IntList.java

Util.java

```java
import java.io.BufferedReader;
import java.io.IOException;

public class Util {
    public static int readInt(BufferedReader br, String prompt) throws
        IOException {
        while (true) {
            System.out.print(prompt);
            String line = br.readLine();
            if (line == null) { System.out.println(); System.exit(0); }
            try { return Integer.parseInt(line.trim()); }
            catch (NumberFormatException e) { System.out.println("Please enter
                a whole number."); }
        }
    }

    // Reads a line of space-separated integers (blank line -> empty array)
    public static int[] readInts(BufferedReader br, String prompt) throws
        IOException {
        System.out.print(prompt);
        String line = br.readLine();
        if (line == null || line.trim().isEmpty()) return new int[0];
        String[] parts = line.trim().split("\\s+");
        IntList list = new IntList();
        for (String p : parts) {
            try { list.add(Integer.parseInt(p)); } catch
                (NumberFormatException e) { }
        }
        return list.toArray();
    }
}
```

Fig. A.9 Source code of Util.java

FileSearcher.java (part 1 of 4)

```java
import java.io.BufferedReader;
import java.io.IOException;

// Universal search: crop name, disease, pest, soil, fertilizer... anything in the corpus.
// Uses Z-algorithm (exact crop name), KMP (search inside every field), Edit Distance (fuzzy).
public class FileSearcher {

    public static void search(KnowledgeBase kb, String query, BufferedReader br) throws IOException {
        String q = query.trim().toLowerCase();
        System.out.println("\n========== SEARCH RESULTS ==========");
        if (q.isEmpty()) { System.out.println("Please type something to search."); return; }

        // 1. Exact crop-name match (Z algorithm) -> show full details
        for (int r = 0; r < kb.count; r++) {
            String name = kb.records[r].name.toLowerCase();
            if (name.length() == q.length() && ZAlgorithm.findAll(name, q).length > 0) {
                printDetails(kb.records[r]);
                return;
            }
        }

        // 2. Search every field of every crop (KMP)
        int[] hits = printHits(kb, q);

        // 3. Nothing found -> fuzzy match with edit distance
        if (hits.length == 0) {
            String s = suggest(kb, q);
            if (s == null) { System.out.println("No related crop, disease or keyword found for \"" + query + "\"."); return; }
            System.out.println("No exact match for \"" + query + "\". Did you mean: \"" + s + "\"?");
```

Fig. A.10 Source code of FileSearcher.java (part 1 of 4)

FileSearcher.java (part 2 of 4)

```
30              for (int r = 0; r < kb.count; r++) {
31                  if (kb.records[r].name.equalsIgnoreCase(s)) {
                        printDetails(kb.records[r]); return; }
32              }
33              hits = printHits(kb, s.toLowerCase());
34          }
35
36          if (hits.length > 0) {
37              int pick = Util.readInt(br, "\nEnter result number for full crop
                    details (0 to go back): ");
38              if (pick >= 1 && pick <= hits.length)
                    printDetails(kb.records[hits[pick - 1]]);
39          }
40      }
41
42      // ---- prints every crop whose fields contain the term; returns record
            indexes
43      private static int[] printHits(KnowledgeBase kb, String q) {
44          IntList hits = new IntList();
45          for (int r = 0; r < kb.count; r++) {
46              CropRecord rec = kb.records[r];
47              for (int f = 0; f < rec.fieldCount; f++) {
48                  if (containsWord(rec.values[f].toLowerCase(), q)) {
                        hits.add(r); break; }
49              }
50          }
51          if (hits.size() == 0) return hits.toArray();
52
53          System.out.println("\"" + q + "\" found in " + hits.size() + "
                crop(s):");
54          for (int h = 0; h < hits.size(); h++) {
55              CropRecord rec = kb.records[hits.get(h)];
56              System.out.println("\n " + (h + 1) + ") " + rec.name);
57              for (int f = 0; f < rec.fieldCount; f++) {
58                  if (KnowledgeBase.isNameKey(rec.keys[f])) continue;
59                  if (containsWord(rec.values[f].toLowerCase(), q)) {
```

Fig. A.11 Source code of FileSearcher.java (part 2 of 4)

FileSearcher.java (part 3 of 4)

```
60                      System.out.println("      - " + rec.keys[f] + ": " +
                            rec.values[f]);
61                  }
62              }
63          }
64          return hits.toArray();
65      }
66
67      // KMP match that must start and end on a word boundary
68      private static boolean containsWord(String text, String q) {
69          int[] pos = KMP.findAll(text, q);
70          for (int i = 0; i < pos.length; i++) {
71              int s = pos[i], e = s + q.length();
72              boolean left = s == 0 || !isWordChar(text.charAt(s - 1));
73              boolean right = e == text.length() || !isWordChar(text.charAt(e));
74              if (left && right) return true;
75          }
76          return false;
77      }
78
79      private static boolean isWordChar(char c) { return
            Character.isLetterOrDigit(c); }
80
81      // ---- fuzzy suggestion: closest crop name / word in the corpus (Edit
            Distance)
82      private static String suggest(KnowledgeBase kb, String q) {
83          if (q.length() < 3) return null;
84          int limit = q.length() <= 4 ? 1 : 2;
85          String best = null;
86          int bestD = Integer.MAX_VALUE;
87
88          for (int r = 0; r < kb.count; r++) {
89              CropRecord rec = kb.records[r];
90              // whole crop name (handles multi-word names)
91              int d = distanceIfClose(q, rec.name.toLowerCase(), limit);
92              if (d >= 0 && d < bestD) { bestD = d; best = rec.name; }
```

Fig. A.12 Source code of FileSearcher.java (part 3 of 4)

FileSearcher.java (part 4 of 4)

```
            // every word in every field
            for (int f = 0; f < rec.fieldCount; f++) {
                String v = rec.values[f];
                int i = 0;
                while (i < v.length()) {
                    while (i < v.length() && !isWordChar(v.charAt(i))) i++;
                    int s = i;
                    while (i < v.length() && isWordChar(v.charAt(i))) i++;
                    if (i > s) {
                        String w = v.substring(s, i);
                        int dw = distanceIfClose(q, w.toLowerCase(), limit);
                        if (dw >= 0 && dw < bestD) { bestD = dw; best = w; }
                    }
                }
            }
        }
        return best;
    }

    private static int distanceIfClose(String a, String b, int limit) {
        if (Math.abs(a.length() - b.length()) > limit || b.length() < 3)
            return -1;
        int d = EditDistance.calculate(a, b);
        return d <= limit ? d : -1;
    }

    private static void printDetails(CropRecord rec) {
        System.out.println("\n========== CROP DETAILS ==========");
        for (int f = 0; f < rec.fieldCount; f++) {
            System.out.println(rec.keys[f] + ": " + rec.values[f]);
        }
        System.out.println("==================================");
    }
}
```

Fig. A.13 Source code of FileSearcher.java (part 4 of 4)

NaiveSearch.java

```
// CO1 - Naive pattern matching, O(n*m)
public class NaiveSearch {
    public static long comparisons = 0;

    public static int[] findAll(String text, String pat) {
        IntList r = new IntList();
        int n = text.length(), m = pat.length();
        if (m == 0) return r.toArray();
        for (int i = 0; i + m <= n; i++) {
            int j = 0;
            while (j < m) {
                comparisons++;
                if (text.charAt(i + j) != pat.charAt(j)) break;
                j++;
            }
            if (j == m) r.add(i);
        }
        return r.toArray();
    }
}
```

Fig. A.14 Source code of NaiveSearch.java

KMP.java (part 1 of 2)

```java
// CO2 - Knuth-Morris-Pratt, O(n + m)
public class KMP {
    public static long comparisons = 0;

    public static int[] buildLPS(String p) {
        int[] lps = new int[p.length()];
        int len = 0, i = 1;
        while (i < p.length()) {
            if (p.charAt(i) == p.charAt(len)) {
                lps[i++] = ++len;
            } else if (len > 0) {
                len = lps[len - 1];
            } else {
                lps[i++] = 0;
            }
        }
        return lps;
    }

    public static int[] findAll(String text, String pat) {
        IntList r = new IntList();
        int n = text.length(), m = pat.length();
        if (m == 0) return r.toArray();
        int[] lps = buildLPS(pat);
        int i = 0, j = 0;
        while (i < n) {
            comparisons++;
            if (text.charAt(i) == pat.charAt(j)) {
                i++; j++;
                if (j == m) { r.add(i - j); j = lps[j - 1]; }
            } else if (j > 0) {
                j = lps[j - 1];
            } else {
                i++;
            }
        }
```

Fig. A.15 Source code of KMP.java (part 1 of 2)

KMP.java (part 2 of 2)

```java
        return r.toArray();
    }
}
```

Fig. A.16 Source code of KMP.java (part 2 of 2)

ZAlgorithm.java

```java
// CO2 - Z-function pattern matching, O(n + m)
public class ZAlgorithm {

    public static int[] calculateZ(String text) {
        int n = text.length();
        int[] z = new int[n];
        int left = 0, right = 0;
        for (int i = 1; i < n; i++) {
            if (i <= right) z[i] = Math.min(right - i + 1, z[i - left]);
            while (i + z[i] < n && text.charAt(z[i]) == text.charAt(i + z[i]))
                z[i]++;
            if (i + z[i] - 1 > right) { left = i; right = i + z[i] - 1; }
        }
        return z;
    }

    // Returns every start position of pattern in text
    public static int[] findAll(String text, String pattern) {
        IntList r = new IntList();
        if (pattern.length() == 0) return r.toArray();
        int[] z = calculateZ(pattern + "\u0001" + text);
        for (int i = pattern.length() + 1; i < z.length; i++) {
            if (z[i] >= pattern.length()) r.add(i - pattern.length() - 1);
        }
        return r.toArray();
    }

    // Kept from the original version
    public static void search(String text, String pattern) {
        int[] pos = findAll(text, pattern);
        for (int p : pos) System.out.println("Match found at position: " + p);
        System.out.println("Total matches: " + pos.length);
    }
}
```

Fig. A.17 Source code of ZAlgorithm.java

RabinKarp.java (part 1 of 2)

```java
// CO2 - Rabin-Karp with polynomial rolling hash and DOUBLE hashing
public class RabinKarp {
    private static final long BASE = 256;
    private static final long M1 = 1000000007L;
    private static final long M2 = 998244353L;

    public static int[] findAll(String text, String pat) {
        IntList r = new IntList();
        int n = text.length(), m = pat.length();
        if (m == 0 || m > n) return r.toArray();

        long p1 = 1, p2 = 1;                        // BASE^(m-1) under each modulus
        for (int i = 0; i < m - 1; i++) { p1 = p1 * BASE % M1; p2 = p2 * BASE % M2; }

        long hp1 = 0, hp2 = 0, ht1 = 0, ht2 = 0;
        for (int i = 0; i < m; i++) {
            hp1 = (hp1 * BASE + pat.charAt(i)) % M1;
            hp2 = (hp2 * BASE + pat.charAt(i)) % M2;
            ht1 = (ht1 * BASE + text.charAt(i)) % M1;
            ht2 = (ht2 * BASE + text.charAt(i)) % M2;
        }
        for (int i = 0; i + m <= n; i++) {
            if (hp1 == ht1 && hp2 == ht2) {         // verify to rule out collisions
                int j = 0;
                while (j < m && text.charAt(i + j) == pat.charAt(j)) j++;
                if (j == m) r.add(i);
            }
            if (i + m < n) {
                ht1 = ((ht1 - text.charAt(i) * p1 % M1 + M1) % M1 * BASE + text.charAt(i + m)) % M1;
                ht2 = ((ht2 - text.charAt(i) * p2 % M2 + M2) % M2 * BASE + text.charAt(i + m)) % M2;
            }
```

Fig. A.18 Source code of RabinKarp.java (part 1 of 2)

RabinKarp.java (part 2 of 2)

```java
        }
        return r.toArray();
    }
}
```

Fig. A.19 Source code of RabinKarp.java (part 2 of 2)

EditDistance.java

```java
// CO3 - Wagner-Fischer (Levenshtein) edit distance
public class EditDistance {
    public static int calculate(String word1, String word2) {
        int m = word1.length(), n = word2.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 0; i <= m; i++) dp[i][0] = i;
        for (int j = 0; j <= n; j++) dp[0][j] = j;
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1];
                } else {
                    dp[i][j] = 1 + Math.min(dp[i - 1][j], Math.min(dp[i][j -
                        1], dp[i - 1][j - 1]));
                }
            }
        }
        return dp[m][n];
    }
}
```

Fig. A.20 Source code of EditDistance.java

Benchmark.java (part 1 of 2)

```java
// CO1 - Algorithm selection: compare Naive, KMP, Z, Rabin-Karp on the real
    corpus
public class Benchmark {
    public static void run(KnowledgeBase kb, String pattern) {
        String text = kb.corpusLower;
        String p = pattern.toLowerCase();
        System.out.println("\n========== STRING SEARCH BENCHMARK ==========");
        System.out.println("Corpus size: " + text.length() + " chars, pattern:
             \"" + p + "\"\n");
        System.out.println(pad("Algorithm", 14) + pad("Matches", 10) +
            pad("Time (us)", 12) + pad("Comparisons", 13) + "Complexity");

        NaiveSearch.comparisons = 0;
        long t = System.nanoTime();
        int a = NaiveSearch.findAll(text, p).length;
        row("Naive", a, System.nanoTime() - t, NaiveSearch.comparisons + "",
            "O(n*m)");

        KMP.comparisons = 0;
        t = System.nanoTime();
        a = KMP.findAll(text, p).length;
        row("KMP", a, System.nanoTime() - t, KMP.comparisons + "", "O(n+m)");

        t = System.nanoTime();
        a = ZAlgorithm.findAll(text, p).length;
        row("Z-Algorithm", a, System.nanoTime() - t, "-", "O(n+m)");

        t = System.nanoTime();
        a = RabinKarp.findAll(text, p).length;
        row("Rabin-Karp", a, System.nanoTime() - t, "-", "O(n+m) avg");
        System.out.println("\nAll four must report the same match count
            (correctness check).");
    }

    private static void row(String name, int matches, long ns, String cmp,
        String cx) {
```

Fig. A.21 Source code of Benchmark.java (part 1 of 2)

Benchmark.java (part 2 of 2)

```
31          System.out.println(pad(name, 14) + pad(matches + "", 10) + pad((ns /
                1000) + "", 12) + pad(cmp, 13) + cx);
32      }
33
34      private static String pad(String s, int w) {
35          while (s.length() < w) s += " ";
36          return s;
37      }
38  }
```

Fig. A.22 Source code of Benchmark.java (part 2 of 2)

MaxFlow.java (part 1 of 3)

```
1   import java.io.BufferedReader;
2   import java.io.IOException;
3
4   // CO4 - Edmonds-Karp (Ford-Fulkerson with BFS)
5   public class MaxFlow {
6
7       private int vertices;
8       private int[][] lastResidual;
9
10      public MaxFlow(int vertices) { this.vertices = vertices; }
11
12      public int[][] getResidual() { return lastResidual; }
13
14      private boolean bfs(int[][] residual, int source, int destination, int[]
            parent) {
15          boolean[] visited = new boolean[vertices];
16          int[] queue = new int[vertices];
17          int front = 0, rear = 0;
18          queue[rear++] = source;
19          visited[source] = true;
20          parent[source] = -1;
21          while (front < rear) {
22              int current = queue[front++];
23              for (int next = 0; next < vertices; next++) {
24                  if (!visited[next] && residual[current][next] > 0) {
25                      queue[rear++] = next;
26                      parent[next] = current;
27                      visited[next] = true;
28                      if (next == destination) return true;
29                  }
30              }
31          }
32          return false;
33      }
34
35      public int findMaxFlow(int[][] graph, int source, int destination) {
```

Fig. A.23 Source code of MaxFlow.java (part 1 of 3)

MaxFlow.java (part 2 of 3)

```
36          int[][] residual = new int[vertices][vertices];
37          for (int i = 0; i < vertices; i++)
38              for (int j = 0; j < vertices; j++) residual[i][j] = graph[i][j];
39
40          int[] parent = new int[vertices];
41          int maxFlow = 0;
42          while (bfs(residual, source, destination, parent)) {
43              int pathFlow = Integer.MAX_VALUE;
44              for (int v = destination; v != source; v = parent[v])
45                  pathFlow = Math.min(pathFlow, residual[parent[v]][v]);
46              for (int v = destination; v != source; v = parent[v]) {
47                  residual[parent[v]][v] -= pathFlow;
48                  residual[v][parent[v]] += pathFlow;
49              }
50              maxFlow += pathFlow;
51          }
52          lastResidual = residual;
53          return maxFlow;
54      }
55
56      // Water distribution: any network of pipes
57      public static void runWaterDistribution(BufferedReader br) throws
            IOException {
58          System.out.println("\n========== WATER DISTRIBUTION (Max Flow)
                ==========");
59          int n = Util.readInt(br, "Number of locations (numbered 0..n-1): ");
60          int source = Util.readInt(br, "Source location (e.g. reservoir): ");
61          int dest = Util.readInt(br, "Destination location (e.g. field): ");
62          int pipes = Util.readInt(br, "Number of pipes: ");
63          if (source < 0 || source >= n || dest < 0 || dest >= n) {
                System.out.println("Invalid locations."); return; }
64
65          int[][] graph = new int[n][n];
66          for (int i = 1; i <= pipes; i++) {
67              int[] e = Util.readInts(br, "Pipe " + i + " -> from to
                    capacity(litres/min): ");
```

Fig. A.24 Source code of MaxFlow.java (part 2 of 3)

MaxFlow.java (part 3 of 3)

```
68              if (e.length < 3 || e[0] < 0 || e[0] >= n || e[1] < 0 || e[1] >=
                    n) { System.out.println("  Skipped (bad input)."); continue; }
69              graph[e[0]][e[1]] += e[2];
70          }
71          int result = new MaxFlow(n).findMaxFlow(graph, source, dest);
72          System.out.println("Maximum Flow: " + result + " litres/minute");
73      }
74  }
```

Fig. A.25 Source code of MaxFlow.java (part 3 of 3)

CropFieldMatcher.java (part 1 of 2)

```java
import java.io.BufferedReader;
import java.io.IOException;

// CO4 - Bipartite matching solved as a max-flow problem: assign crops to fields
public class CropFieldMatcher {
    public static void run(BufferedReader br, KnowledgeBase kb) throws IOException {
        System.out.println("\n========== CROP -> FIELD ASSIGNMENT (Bipartite Matching) ==========");
        int L = Math.min(kb.count, 8);
        if (L == 0) { System.out.println("No crops loaded."); return; }
        int F = Util.readInt(br, "Number of fields available: ");
        if (F <= 0) return;

        int V = L + F + 2, S = 0, T = V - 1;
        int[][] g = new int[V][V];
        for (int i = 0; i < L; i++) {
            g[S][1 + i] = 1;
            int[] ok = Util.readInts(br, "Fields (1.." + F + ") suitable for "
                    + kb.records[i].name + " (space separated): ");
            for (int f : ok) if (f >= 1 && f <= F) g[1 + i][1 + L + (f - 1)] =
                    1;
        }
        for (int j = 0; j < F; j++) g[1 + L + j][T] = 1;

        MaxFlow mf = new MaxFlow(V);
        int matched = mf.findMaxFlow(g, S, T);
        int[][] res = mf.getResidual();

        System.out.println("\nCrops that can be planted at the same time: " +
            matched);
        for (int i = 0; i < L; i++) {
            String field = "not assigned";
            for (int j = 0; j < F; j++)
                if (g[1 + i][1 + L + j] == 1 && res[1 + i][1 + L + j] == 0)
```

Fig. A.26 Source code of CropFieldMatcher.java (part 1 of 2)

CropFieldMatcher.java (part 2 of 2)

```java
                    field = "Field " + (j + 1);
            System.out.println("  " + kb.records[i].name + " -> " + field);
        }
    }
}
```

Fig. A.27 Source code of CropFieldMatcher.java (part 2 of 2)

RoutePlanner.java (part 1 of 2)

```java
import java.io.BufferedReader;
import java.io.IOException;

// CO3 - Bitmask DP (Travelling Salesperson): shortest route visiting all fields/markets
public class RoutePlanner {
    public static void run(BufferedReader br) throws IOException {
        System.out.println("\n========== FIELD VISIT ROUTE PLANNER (Bitmask DP / TSP) ==========");
        int n = Util.readInt(br, "Number of locations (location 0 = farm, max 15): ");
        if (n < 2 || n > 15) { System.out.println("Please use between 2 and 15 locations."); return; }
        int[][] d = new int[n][n];
        for (int i = 0; i < n; i++) {
            int[] row = Util.readInts(br, "Distances from location " + i + " to 0.." + (n - 1) + ": ");
            for (int j = 0; j < n && j < row.length; j++) d[i][j] = row[j];
        }

        final int INF = 1000000000;
        int FULL = 1 << n;
        int[][] dp = new int[FULL][n];
        int[][] par = new int[FULL][n];
        for (int m = 0; m < FULL; m++) for (int i = 0; i < n; i++) { dp[m][i] = INF; par[m][i] = -1; }
        dp[1][0] = 0;

        for (int mask = 1; mask < FULL; mask++) {
            for (int i = 0; i < n; i++) {
                if ((mask & (1 << i)) == 0 || dp[mask][i] >= INF) continue;
                for (int j = 0; j < n; j++) {
                    if ((mask & (1 << j)) != 0) continue;
                    int nm = mask | (1 << j);
                    int cost = dp[mask][i] + d[i][j];
                    if (cost < dp[nm][j]) { dp[nm][j] = cost; par[nm][j] = i;
```

Fig. A.28 Source code of RoutePlanner.java (part 1 of 2)

RoutePlanner.java (part 2 of 2)

```java
                    }
                }
            }
        }
        int best = INF, last = -1;
        for (int i = 1; i < n; i++) {
            if (dp[FULL - 1][i] < INF && dp[FULL - 1][i] + d[i][0] < best) {
                best = dp[FULL - 1][i] + d[i][0]; last = i; }
        }
        int[] path = new int[n];
        int mask = FULL - 1, cur = last;
        for (int k = n - 1; k >= 1; k--) { path[k] = cur; int p = par[mask][cur]; mask ^= (1 << cur); cur = p; }
        path[0] = 0;

        StringBuilder sb = new StringBuilder("0");
        for (int k = 1; k < n; k++) sb.append(" -> ").append(path[k]);
        sb.append(" -> 0");
        System.out.println("Shortest round trip: " + sb);
        System.out.println("Total distance: " + best);
    }
}
```

Fig. A.29 Source code of RoutePlanner.java (part 2 of 2)

FieldInspector.java (part 1 of 2)

```java
import java.io.BufferedReader;
import java.io.IOException;

// CO5 - Vertex Cover 2-approximation (maximal matching) vs exact brute force.
// Fields are vertices; an edge = two neighbouring fields that can pass a disease to each other.
// Inspecting a "vertex cover" of fields covers every risky neighbour pair.
public class FieldInspector {
    public static void run(BufferedReader br) throws IOException {
        System.out.println("\n========== FIELD INSPECTION PLANNER (Vertex Cover 2-Approximation) ==========");
        int n = Util.readInt(br, "Number of fields (0..n-1, max 20): ");
        if (n < 1 || n > 20) { System.out.println("Please use 1 to 20 fields."); return; }
        int m = Util.readInt(br, "Number of neighbouring pairs: ");
        int[] a = new int[m], b = new int[m];
        int cnt = 0;
        for (int i = 1; i <= m; i++) {
            int[] e = Util.readInts(br, "Pair " + i + " (two field numbers): ");
            if (e.length >= 2 && e[0] >= 0 && e[0] < n && e[1] >= 0 && e[1] < n) { a[cnt] = e[0]; b[cnt] = e[1]; cnt++; }
        }

        // 2-approximation: take both endpoints of any uncovered edge
        boolean[] chosen = new boolean[n];
        for (int i = 0; i < cnt; i++) {
            if (!chosen[a[i]] && !chosen[b[i]]) { chosen[a[i]] = true; chosen[b[i]] = true; }
        }
        int approx = 0;
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) if (chosen[i]) { approx++; sb.append(i).append(' '); }

        // exact optimum by trying every subset (fine for n <= 20)
```

Fig. A.30 Source code of FieldInspector.java (part 1 of 2)

FieldInspector.java (part 2 of 2)

```java
        int opt = n;
        for (int mask = 0; mask < (1 << n); mask++) {
            int size = Integer.bitCount(mask);
            if (size >= opt) continue;
            boolean ok = true;
            for (int i = 0; i < cnt && ok; i++)
                if ((mask & (1 << a[i])) == 0 && (mask & (1 << b[i])) == 0) ok = false;
            if (ok) opt = size;
        }
        System.out.println("Fields to inspect (approximation): " + sb.toString().trim() + "  [" + approx + " fields]");
        System.out.println("Optimal number of fields (exact):  " + opt);
        if (opt > 0) System.out.println("Approximation ratio: " + (approx * 1.0 / opt) + "  (guaranteed <= 2)");
    }
}
```

Fig. A.31 Source code of FieldInspector.java (part 2 of 2)

ReservoirSampler.java

```java
// CO6 - Reservoir sampling: uniform random k items from a stream of unknown length.
// Used for "Crops of the day". Own random generator (java.util.Random is not allowed).
public class ReservoirSampler {
    private static long seed = System.nanoTime();

    private static int nextInt(int bound) {
        seed = seed * 6364136223846793005L + 1442695040888963407L;
        return (int) ((seed >>> 33) % bound);
    }

    public static String[] sample(String[] stream, int total, int k) {
        if (k > total) k = total;
        String[] reservoir = new String[k];
        for (int i = 0; i < total; i++) {
            if (i < k) {
                reservoir[i] = stream[i];
            } else {
                int j = nextInt(i + 1);          // 0..i
                if (j < k) reservoir[j] = stream[i];
            }
        }
        return reservoir;
    }
}
```

Fig. A.32 Source code of ReservoirSampler.java